

Equipo:

Miguel Figuera C.I: 23.558.789 Iromy Leon C.I: V-30.243.131 Alejandra Herde C.I: V-23.711.974

Link del repositorio:

<https://github.com/miguelfiguera/vet-crm-platform>

Sistema de Autenticación - PawCare

Este documento describe el sistema de autenticación implementado en PawCare, incluyendo JWT, manejo de tokens, recuperación de contraseña y configuración de emails.

Tabla de Contenidos

- [Visión General](#)
- [Gemas Relacionadas](#)
- [Duración de Tokens](#)
- [Arquitectura del Sistema](#)
- [Flujos de Autenticación](#)
- [Cambio de Contraseña](#)
- [Recuperación de Contraseña](#)
- [Creación de Usuarios Admin](#)
- [Mails](#)
- [Letter Opener Web](#)
- [Seeders](#)
- [Archivos Clave](#)

Visión General

PawCare utiliza un sistema de autenticación basado en **JWT (JSON Web Tokens)** con las siguientes características:

- **Cookies httpOnly:** Los tokens se almacenan en cookies encriptadas, previniendo ataques XSS
- **Soporte polimórfico:** Soporta dos tipos de usuarios: **Owner** (clientes) y **User** (staff/admin)
- **Token Rotation:** Los refresh tokens se invalidan al usarse y se genera uno nuevo
- **Soft Delete:** Las cuentas se "eliminan" mediante soft delete (gem **discard**)

Tipos de Usuario

Modelo	Descripción	Roles
Owner	Dueños de mascotas (clientes)	N/A
User	Personal de la veterinaria	admin, vet, vet_assistant, assistant, finances, hr

Gemas Relacionadas

```
# Gemfile

# Autenticación
gem "bcrypt", "~> 3.1.7"           # Hash de contraseñas
(has_secure_password)
gem "jwt", "~> 2.9"                # Encoding/decoding de JWT

# Soft Delete
gem "discard", "~> 1.4"           # Soft delete para cuentas

# Emails (desarrollo)
gem "letter_opener", "~> 1.10"    # Abre emails en el navegador
gem "letter_opener_web", "~> 3.0" # UI web para ver emails
```

Duración de Tokens

Los tokens tienen las siguientes duraciones configuradas en `app/services/jwt_service.rb`:

Token	Duración	Ubicación
Access Token	1 hora	Cookie encriptada <code>access_token</code>
Refresh Token	7 días	Cookie encriptada <code>refresh_token</code> + BD
Password Reset Token	1 hora	Base de datos (tabla <code>password_reset_tokens</code>)

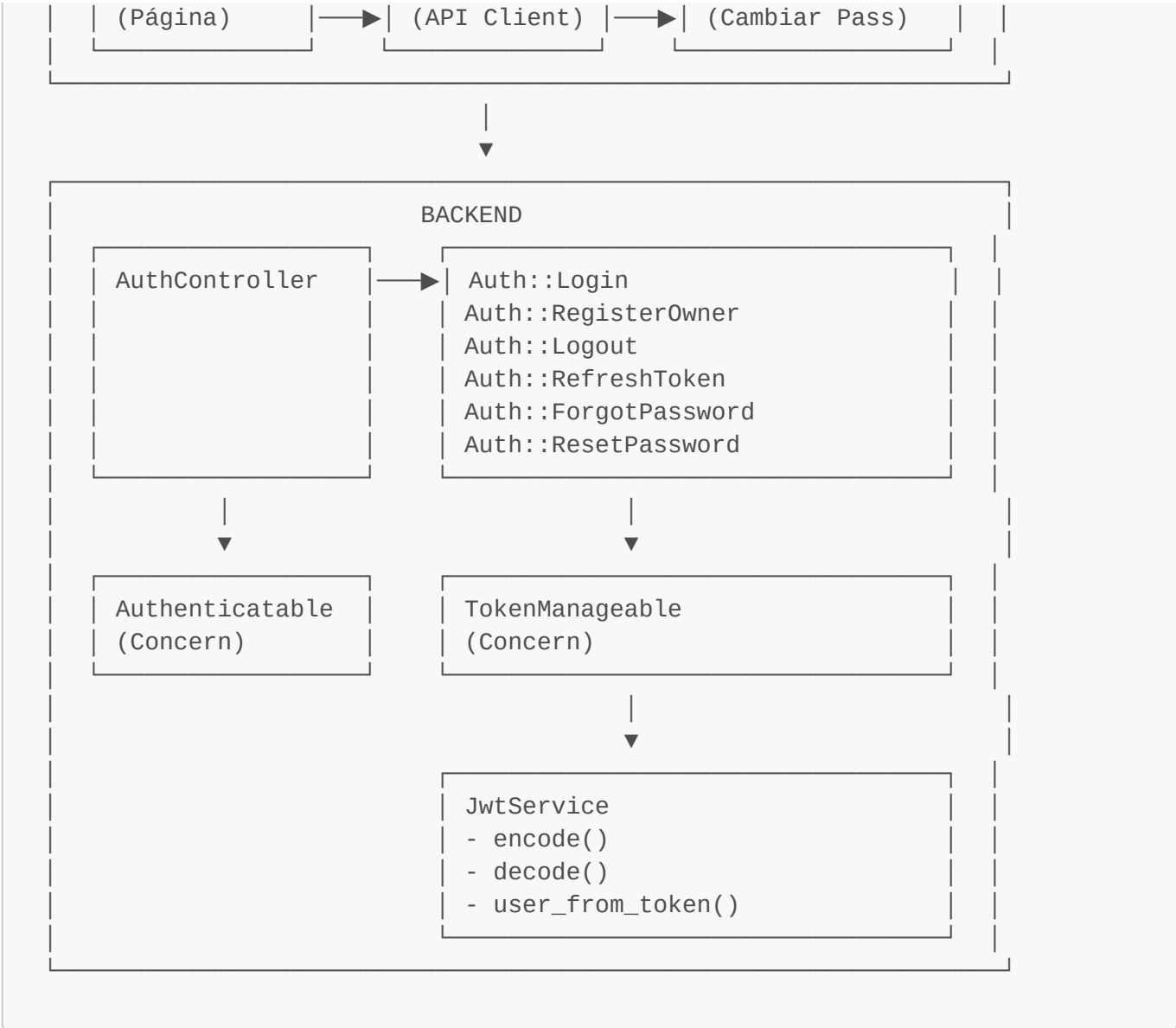
```
# app/services/jwt_service.rb
ACCESS_TOKEN_EXPIRATION = 1.hour
REFRESH_TOKEN_EXPIRATION = 7.days

# app/actions/auth/forgot_password.rb
PasswordResetToken.create!(
  expires_at: 1.hour.from_now
)
```

Arquitectura del Sistema

Diagrama de Componentes





Modelos de Datos

```
# RefreshToken (tabla: refresh_tokens)
# - token: string (único, valor del token)
# - tokenable_type: string (polimórfico: "Owner" o "User")
# - tokenable_id: bigint (ID del usuario)
# - expires_at: datetime (expiración)
# - revoked_at: datetime (null si activo)

# PasswordResetToken (tabla: password_reset_tokens)
# - token: string (único, valor del token)
# - resettable_type: string (polimórfico: "Owner" o "User")
# - resettable_id: bigint (ID del usuario)
# - expires_at: datetime (expiración)
# - used_at: datetime (null si no usado)
```

Flujos de Autenticación

1. Login



Archivo: `app/actions/auth/login.rb`

```
# Busca en ambos modelos por email o username
def find_authenticatable
  login = @params[:login]&.downcase
  Owner.find_by(email: login) ||
  Owner.find_by(username: login) ||
  User.find_by(email: login) ||
  User.find_by(username: login)
end
```

2. Registro de Owner



1. Crea Owner
2. Genera tokens
3. Set cookies
4. Envía email de bienvenida



Usuario autenticado automáticamente

Archivo: `app/actions/auth/register_owner.rb`

3. Refresh de Tokens

Access token expirado (1 hora)



POST /auth/refresh
(con cookie refresh_token)



Auth::RefreshToken.new.call
1. Valida refresh token
2. REVOCA token anterior
3. Genera nuevos tokens
4. Set nuevas cookies



Sesión renovada por 7 días más

Archivo: `app/actions/auth/refresh_token.rb`

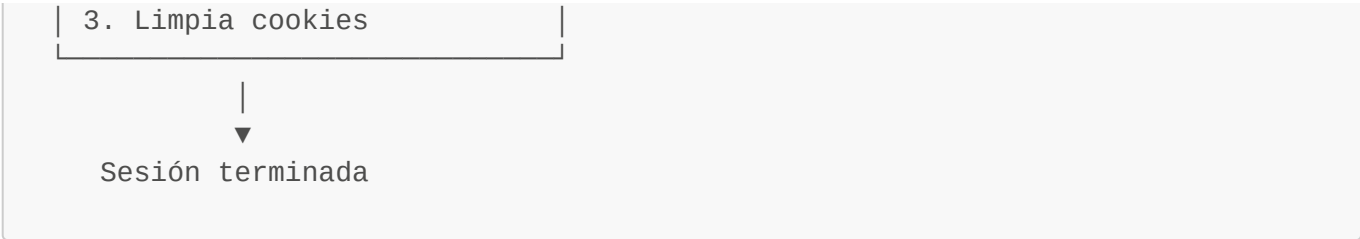
Nota: Se implementa **Token Rotation** - el refresh token usado se invalida inmediatamente.

4. Logout

DELETE /auth/logout
(requiere autenticación)



Auth::Logout.new.call
1. Busca refresh token
2. Revoca (revoked_at)



Archivo: `app/actions/auth/logout.rb`

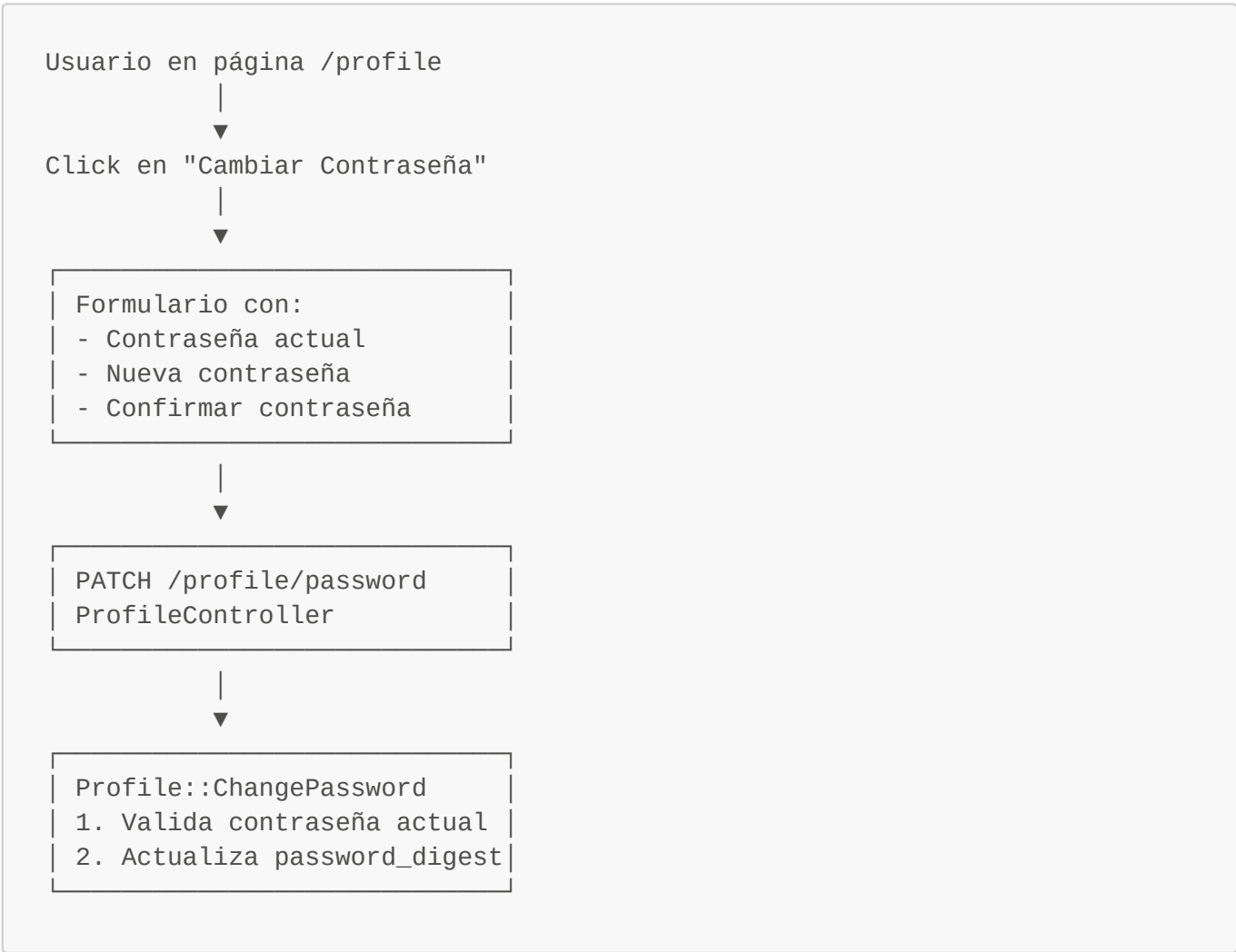
Cambio de Contraseña

El cambio de contraseña está disponible para usuarios autenticados (tanto Owners como Staff).

Ubicación en la UI

Tipo de Usuario	Ruta	Navegación
Owner	<code>/profile</code>	Header → Avatar → "Ver perfil" → "Cambiar Contraseña"
Staff/Admin	<code>/profile</code>	Header → Avatar → "Ver perfil" → "Cambiar Contraseña"

Flujo



- Backend: `app/actions/profile/change_password.rb`
- Frontend: `app/frontend/pages/profile/ProfileIndex.tsx`

Recuperación de Contraseña

Flujo Completo

1. Usuario hace click en "¿Olvidaste tu contraseña?"



2. Ingresas su email



```
POST /auth/forgot_password
Auth::ForgotPassword
1. Busca Owner o User
2. Crea PasswordResetToken
3. Envía email con link
```



3. Usuario recibe email con link:
`/auth/reset_password/{token}`



4. Usuario hace click en el link



```
GET /auth/reset_password/:token |
Renderiza formulario
```



5. Usuario ingresa nueva contraseña



```
POST /auth/reset_password
Auth::ResetPassword
1. Valida token activo
2. Actualiza contraseña
3. Marca token como usado
```



6. Usuario puede iniciar sesión con nueva contraseña

Archivos:

- `app/actions/auth/forgot_password.rb`
- `app/actions/auth/reset_password.rb`
- `app/models/password_reset_token.rb`

Seguridad: El endpoint siempre retorna éxito para no revelar si el email existe.

Creación de Usuarios Admin

Actualmente **no existe UI** para crear usuarios Admin/Staff. Se debe hacer via Rails console.

Via Rails Console

```
# Abrir consola
bin/rails console

# Crear admin
User.create!(
  first_name: "Nuevo",
  last_name: "Admin",
  identity_document: "V12345678",
  email: "nuevo.admin@pawcare.com",
  username: "nuevoadmin",
  password: "password123",
  role: :admin,
  active: true
)

# Crear veterinario
User.create!(
  first_name: "Dr. Juan",
  last_name: "Pérez",
  identity_document: "V87654321",
  email: "juan.perez@pawcare.com",
  username: "drjuan",
  password: "password123",
  role: :vet,
  active: true
)
```

Via Action (para futura implementación)

```
# Usar el action existente
result = Users::CreateUser.new(
  params: {
    first_name: "Nuevo",
    last_name: "Admin",
    identity_document: "V12345678",
    email: "nuevo.admin@pawcare.com",
    username: "nuevoadmin",
```



```
        password: "password123"
      },
      role: :admin
    ).call

    if result.success?
      puts "Usuario creado: #{result.user.email}"
    else
      puts "Errores: #{result.errors}"
    end
  end
end
```

Roles Disponibles

```
enum :role, {
  admin: 0,      # Administrador completo
  vet: 1,        # Veterinario
  vet_assistant: 2, # Asistente de veterinario
  assistant: 3,  # Asistente general
  finances: 4,   # Personal de finanzas
  hr: 5          # Recursos humanos
}
```

Mailers

Mailers de Autenticación

Mailer	Método	Descripción
OwnerMailer	welcome(owner)	Email de bienvenida al registrarse
OwnerMailer	password_reset(owner, token)	Link para restablecer contraseña
UserMailer	welcome(user)	Email de bienvenida a staff
UserMailer	password_reset(user, token)	Link para restablecer contraseña

Estructura de Templates

```
app/views/
├── owner_mailer/
│   ├── welcome.html.erb
│   ├── welcome.text.erb
│   ├── password_reset.html.erb
│   └── password_reset.text.erb
└── user_mailer/
    ├── welcome.html.erb
    └── welcome.text.erb
```

```
|— password_reset.html.erb  
|— password_reset.text.erb
```

Contenido del Email de Password Reset

El email incluye:

- Nombre del usuario
- Link con token válido por 1 hora
- Advertencia de expiración
- Instrucciones si no solicitó el cambio

Letter Opener Web

En desarrollo, los emails se pueden visualizar en el navegador usando Letter Opener Web.

Acceso

```
http://localhost:3000/letter_opener
```

Configuración

```
# config/environments/development.rb  
config.action_mailer.delivery_method = :letter_opener_web  
config.action_mailer.perform_deliveries = true  
config.action_mailer.raise_delivery_errors = true  
config.action_mailer.default_url_options = { host: "localhost", port: 3000  
}  
  
# config/routes.rb  
mount LetterOpenerWeb::Engine, at: "/letter_opener" if  
Rails.env.development?
```

Uso

1. Ejecutar una acción que envíe email (registro, forgot password, etc.)
2. Navegar a http://localhost:3000/letter_opener
3. Ver todos los emails enviados en la sesión
4. Click en un email para ver su contenido HTML y texto plano

Seeders

Usuarios de Prueba

Los seeders crean usuarios de prueba para desarrollo. Ejecutar con:

```
bin/rails db:seed
```

db/seeds/users.rb

Crea usuarios staff:

Email	Password	Rol	Nombre
admin@pawcare.com	password123	admin	Admin PawCare
vet@pawcare.com	password123	vet	Dr. María Veterinaria

db/seeds/owners.rb

Crea owners de prueba:

Email	Password	Nombre
owner1@example.com	password123	Juan Pérez
owner2@example.com	password123	María García
owner3@example.com	password123	Carlos López

Resumen de Credenciales

=====

CREDENCIALES DE PRUEBA

=====

STAFF (Dashboard Admin):

- admin@pawcare.com / password123 (Admin)
- vet@pawcare.com / password123 (Veterinario)

OWNERS (Dashboard Cliente):

- owner1@example.com / password123
- owner2@example.com / password123
- owner3@example.com / password123

=====

Archivos Clave

Backend

Archivo	Propósito
---------	-----------

Archivo	Propósito
<code>app/services/jwt_service.rb</code>	Encoding/decoding de JWT tokens
<code>app/actions/concerns/token_manageable.rb</code>	Manejo de cookies y generación de tokens
<code>app/actions/auth/login.rb</code>	Lógica de login
<code>app/actions/auth/register_owner.rb</code>	Registro de owners
<code>app/actions/auth/logout.rb</code>	Cierre de sesión
<code>app/actions/auth/refresh_token.rb</code>	Renovación de tokens
<code>app/actions/auth/forgot_password.rb</code>	Solicitud de reset
<code>app/actions/auth/reset_password.rb</code>	Cambio de contraseña con token
<code>app/actions/profile/change_password.rb</code>	Cambio de contraseña autenticado
<code>app/models/refresh_token.rb</code>	Modelo de refresh tokens
<code>app/models/password_reset_token.rb</code>	Modelo de reset tokens
<code>app/controllers/auth_controller.rb</code>	Endpoints de autenticación
<code>app/controllers/profile_controller.rb</code>	Endpoints de perfil
<code>app/controllers/concerns/authenticatable.rb</code>	Middleware de autenticación

Frontend

Archivo	Propósito
<code>app/frontend/api/Auth.ts</code>	Cliente API de autenticación
<code>app/frontend/api/Profile.ts</code>	Cliente API de perfil
<code>app/frontend/pages/Auth.tsx</code>	Página de login/registro/reset
<code>app/frontend/pages/profile/ProfileIndex.tsx</code>	Página de perfil y cambio de contraseña
<code>app/frontend/components/auth/LoginForm.tsx</code>	Formulario de login
<code>app/frontend/components/auth/RegisterForm.tsx</code>	Formulario de registro
<code>app/frontend/components/auth/ForgotPasswordForm.tsx</code>	Formulario de forgot password
<code>app/frontend/components/auth/ResetPasswordForm.tsx</code>	Formulario de reset password

Tests

Archivo	Propósito
<code>spec/requests/auth_spec.rb</code>	Tests de endpoints de auth
<code>spec/requests/profile_spec.rb</code>	Tests de endpoints de perfil

Archivo	Propósito
<code>spec/support/authentication_helper.rb</code>	Helper para autenticar en tests

Rutas de Autenticación

```
# config/routes.rb

# Auth pages
get "auth", to: "auth#index"
get "auth/reset_password/:token", to: "auth#edit_password_reset"

# Auth API
scope :auth do
  post "register_owner", to: "auth#register_owner"
  post "login", to: "auth#login"
  delete "logout", to: "auth#logout"
  post "refresh", to: "auth#refresh"
  get "current_user", to: "auth#current_user"
  post "forgot_password", to: "auth#forgot_password"
  post "reset_password", to: "auth#reset_password"
end

# Profile API
get "profile", to: "profile#show"
patch "profile", to: "profile#update"
patch "profile/password", to: "profile#change_password"
delete "profile", to: "profile#destroy"
```

Seguridad

Medidas Implementadas

1. **Cookies httpOnly:** Previene acceso desde JavaScript (XSS)
2. **Cookies encriptadas:** Rails encripta el contenido de las cookies
3. **SameSite=Lax:** Protección contra CSRF
4. **Secure en producción:** Cookies solo se envían por HTTPS
5. **Token Rotation:** Refresh tokens se invalidan al usarse
6. **Soft Revocation:** Tokens se marcan como revocados (no eliminados)
7. **Validación de cuenta activa:** Usuarios inactivos no pueden autenticarse
8. **No revelar existencia de email:** Forgot password siempre retorna éxito

Configuración de Cookies

```
# app/actions/concerns/token_manageable.rb
cookies.encrypted[:access_token] = {
  value: access_token,
```

```
  httponly: true,  
  secure: Rails.env.production?,  
  same_site: :lax  
}
```